
RADAR POSTURE CLASSIFICATION: A COMPARATIVE STUDY OF SOTA ARCHITECTURES

STUDENT PAPER

Eloan Tourtelier, Othmane Ouali, Lucas Cheylan, Mellouki Meriem, Tarhda Zaineb

2026

ABSTRACT

We study posture classification from radar point clouds and compare several model architectures for distinguishing between sitting and standing positions. We describe our radar point representation, training protocol, and evaluation setting, and we benchmark multiple baselines under the same experimental conditions. Our results highlight which architectural choices are most effective for sparse radar point data and provide practical guidance for future radar-based human state recognition.

Contents

1	Introduction	2
2	Data	2
2.1	RadHAR dataset	2
2.2	Homemade dataset	3
3	Sketching + Transformer	3
3.1	Preprocessing: Random Fourier Feature sketching	3
3.2	Transformer encoder	4
3.3	Training protocol	4
3.4	Ablation studies	4
4	Attention Pooling	4
5	PointNet	4
6	Moments	4
6.1	Motivation	4
6.2	Preprocessing	5
6.3	Architecture: Multi-Layer Perceptron	6
6.4	Results and Discussion	6
7	XGBoost	6

8 Results	6
8.1 Comparison	6
9 Conclusion	7
A Appendix	7

1 Introduction

2 Data

2.1 RadHAR dataset

The RadHAR dataset, introduced by Singh *et al.* [?] at the 3rd ACM Workshop on Millimeter-Wave Networks and Sensing Systems (mmNets, 2019), is one of the most widely used public benchmarks for human activity recognition (HAR) from millimeter-wave (mmWave) radar point clouds. It is also referred to in the literature as the *MMActivity* dataset.

Acquisition setup. Data was collected with a Texas Instruments IWR1443B00ST mmWave FMCW radar, a low-cost, commercial off-the-shelf sensor operating in the 76–81 GHz band. The radar was mounted on a tripod at a height of 1.3 m, facing the subject. The on-board signal-processing pipeline (through the `radar-lab` ROS package for TI mmWave devices) converts the raw intermediate-frequency signal into a stream of sparse 3D point clouds at a sampling rate of approximately 30 Hz, i.e. one frame every ≈ 33 ms.

Activities and subjects. Two participants performed five activities in front of the radar: boxing, jumping, jumping jacks, squats and walking. The dataset is shipped with a fixed train/test split, provided as separate `train/` and `test/` folders in the official repository. We use this split as-is throughout the paper, so that all our experiments remain comparable with prior work using RadHAR. We note however that the limited number of subjects (only two) is an important caveat: it limits the inter-subject variability seen at training time, and tends to inflate test accuracies compared to what one would expect on a more diverse population.

Structure of a recording. Each recording is a time-ordered sequence of *radar frames*. Two properties make this representation fundamentally different from image or video data and condition every design choice that follows:

- **Sparsity and variable cardinality.** Each frame contains only a small, *variable* number of detected points (typically a few tens), since a low-cost mmWave radar produces sparse and non-uniform point clouds.
- **Unordered point sets.** Within a frame, points have no natural ordering: a frame is a *set*, not a sequence. Any architecture operating directly on raw points should therefore be permutation-invariant at the frame level.

For every detected point, six features are provided:

Feature	Description
x, y, z	Cartesian coordinates of the point (m)
r (range)	Radial distance from the radar (m)
v (velocity)	Doppler radial velocity (m/s)
θ (bearing)	Azimuth angle (rad)
I (intensity)	Reflected signal intensity

The six features are not independent: Cartesian coordinates (x, y, z) , range r and bearing θ are linked by the spherical-to-Cartesian relation, so the raw representation is *redundant*. Which subset of features is actually fed to a model is therefore a non-trivial design choice. The different combinations we explored, and the rationale behind each, are detailed in Section 3.1.

Temporal windowing. Following the original RadHAR protocol, activities are classified over a sliding time window of 2 seconds, which at 30 fps corresponds to a sequence of 60 consecutive frames. Each such window constitutes one labeled sample for the classifier. The resulting object is therefore a *sequence of sets*: 60 unordered, variable-size point sets, collectively carrying a single activity label.

2.2 Homemade dataset

3 Sketching + Transformer

3.1 Preprocessing: Random Fourier Feature sketching

Why preprocessing is needed. A raw radar frame is an *unordered set of points of variable cardinality*. No standard learning architecture — be it an MLP, a CNN, a Transformer or a gradient-boosted tree — can consume such an object directly: they all expect inputs of fixed shape and well-defined structure. A preprocessing step is therefore unavoidable. The original RadHAR baseline addresses this issue through *voxelization*: each 2-second window is discretized into a tensor of shape $60 \times 10 \times 32 \times 32 = 614\,400$ scalar features [?]. This representation is extremely high-dimensional and overwhelmingly sparse, since a typical frame contains only a few tens of points scattered in a grid of 10,240 voxels. The downstream model thus has to invest most of its capacity in processing empty space.

We propose instead to compress each frame into a compact, fixed-size vector via a *random Fourier feature map*, which preserves the information carried by the points while keeping the dimensionality manageable.

Random Fourier feature sketching. For a given choice of d per-point features (see paragraph *Feature selection* below), a frame at time t is a point set $\mathcal{F}_t = \{p_1, \dots, p_{N_t}\} \subset \mathbb{R}^d$, where N_t varies from frame to frame. We map this set to a fixed-size sketch $s_t \in \mathbb{R}^m$ in two steps.

Step 1 — Per-point random projection. We draw a single random matrix $W \in \mathbb{R}^{m \times d}$ with i.i.d. entries

$$W_{ij} \sim \mathcal{N}(0, \sigma^2),$$

once and for all at the beginning of training, and freeze it. Each point p_i is mapped to

$$\varphi(p_i) = \sin(Wp_i) \in \mathbb{R}^m.$$

This is a Fourier-type random feature map (with bias $b = 0$ and using the sine component only), inspired by the random Fourier feature construction of Rahimi and Recht [?]. With $b = 0$ and a single trigonometric component, the inner product $\langle \varphi(p_i), \varphi(p_j) \rangle$ does not strictly approximate a Gaussian kernel; we use it nonetheless as a randomized, non-linear feature map, which proved sufficient for our task.

Step 2 — Aggregation across the frame. Since $\mathcal{F}_t$ is a *set*, the aggregation across its points must be permutation-invariant. We consider two variants:

- **Uniform sketch:** $s_t = \frac{1}{N_t} \sum_{i=1}^{N_t} \sin(Wp_i).$
- **Intensity-weighted sketch:** $s_t = \frac{1}{\sum_i I_i} \sum_{i=1}^{N_t} I_i \sin(Wp_i),$

where I_i denotes the reflected intensity of point i . The intensity-weighted variant gives more importance to high-confidence reflections, which in our experiments proved beneficial.

The whole window is then represented by a tensor of shape $60 \times m$, obtained by stacking the 60 per-frame sketches.

Feature standardization. Per-point features live on widely different scales: range is in metres ($r \in [0, 10]$ approximately), bearing in radians ($\theta \in [-\pi, \pi]$), velocity in m/s, intensity on an arbitrary scale. Because the random projection Wp_i mixes all features linearly, this scale heterogeneity would make the choice of σ ill-defined. We therefore standardize every feature to zero mean and unit variance *before* applying the sketching step. The per-feature mean μ and standard deviation s are computed *on the training set only*, and the same statistics are reused at test time, so that no information leaks from the test set into the preprocessing pipeline.

Feature selection. The six raw features per point $(x, y, z, r, v, \theta, I)$ are redundant: (x, y, z) , r and θ are linked by the spherical-to-Cartesian relation. Determining which subset is most informative is therefore an empirical question. We compared four configurations:

Configuration	Per-point features	Aggregation
All features	$(x, y, z, r, v, \theta, I)$	uniform
Cartesian + I	(x, y, z, I)	uniform
Polar + I	(θ, r, I)	uniform
Polar + I (weighted)	(θ, r, I)	I -weighted

The polar configuration (θ, r, I) consistently outperformed the Cartesian variants, which motivated the introduction of the intensity-weighted sketch as a refinement of that configuration. A detailed comparison is reported in the ablation study (Section 3.4).

Sliding-window construction. Each raw recording is segmented into 2-second windows (60 consecutive frames) using a sliding window with a stride of 10 frames. This yields a controlled overlap (50 frames, $\approx 83\%$) between consecutive windows: small enough to keep them distinct from the classifier’s point of view, large enough to make full use of the available recordings. The procedure produces a total of 12 150 training windows and 3 565 test windows, in line with the official RadHAR train/test split.

Hyperparameter choices. The preprocessing pipeline introduces several hyperparameters whose impact is non-trivial: the random-projection scale σ , the sketch dimension m , the sliding-window stride, the per-point feature subset, and the aggregation scheme (uniform vs. intensity-weighted). All of them were studied empirically; the corresponding ablation study is reported in Section 3.4. Unless stated otherwise, the rest of the paper uses $\sigma = 1$, $m = 256$, polar features (θ, r, I) , intensity-weighted aggregation, and a stride of 10 frames.

Summary. The full preprocessing pipeline can be summarized as

$$\underbrace{\text{recording}}_{\text{raw .txt}} \rightarrow \underbrace{\text{frames}}_{\text{sets, var. size}} \rightarrow \underbrace{\text{2-s windows}}_{60 \text{ frames}} \rightarrow \underbrace{\text{sketched windows}}_{(60, m)} \rightarrow \text{model input.}$$

The data finally fed to all models compared in this paper is therefore a tensor of shape $(12\,150, 60, 256)$ for training and $(3\,565, 60, 256)$ for testing.

3.2 Transformer encoder

3.3 Training protocol

3.4 Ablation studies

4 Attention Pooling

There are two levels at which attention pooling is an interesting proposition:

First

5 PointNet

The PointNet architecture is a fundamental Deep learning architecture applied on radar points, ie unordered sets of points with their three spatial positions. It was first made to classify a cloud of point, and a forking branch of the model can also segment parts of the cloud point to cut it into recognized portions. The architecture is quite straightforward. It has to be point-order invariants as a cloud point doesn't have point ordering. The network uses a specific block called a TNet, which from a hidden layer computes a matrix to apply to that hidden layer, hence making the point representation invariant to rotations, scaling or shearing (which is what we want for a point distribution). Those are the main two features of the PointNet. It was originally trained for a dataset of

6 Moments

6.1 Motivation

The sketching approach described in Section 3 compresses each frame into a fixed-size vector via a random Fourier feature map. A conceptually simpler alternative is to summarize the empirical distribution of points in a frame by its

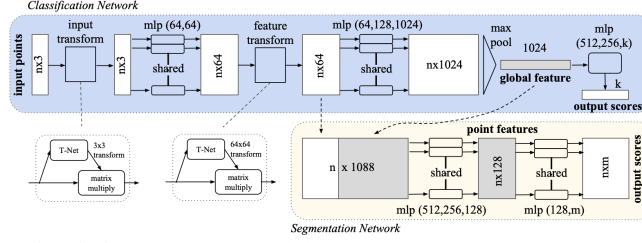


Figure 2. **PointNet Architecture.** The classification network takes n points as input, applies input and feature transformations, and then aggregates point features by max pooling. The output is classification scores for k classes. The segmentation network is an extension to the classification net. It concatenates global and local features and outputs per point scores. “mlp” stands for multi-layer perceptron, numbers in bracket are layer sizes. Batchnorm is used for all layers with ReLU. Dropout layers are used for the last mlp in classification net.

Figure 1: The architecture of a PointNet. The interesting layer is the TNet.

statistical moments: mean, variance, skewness, kurtosis, and range. This representation is fully deterministic, requires no learned or random parameters, and is permutation-invariant by construction.

While statistical moments cannot, in general, uniquely characterize an arbitrary distribution, a classical result in probability theory guarantees that the first $2N$ moments uniquely determine any discrete distribution supported on at most N points [?]. Since each radar frame typically contains fewer than 30 points, a moderate number of moments is therefore *sufficient to characterize the frame exactly*, up to point-set permutation. This property makes the moment representation particularly well-suited to the sparse radar setting, where voxel-based or Fourier-based approaches would introduce unnecessary complexity.

6.2 Preprocessing

Feature selection. Each detected point carries six raw features: x, y, z (Cartesian coordinates), range r , radial velocity v , bearing θ , and intensity I . As noted in Section 2.1, Cartesian coordinates and polar features are geometrically redundant. We found it sufficient to work with a compact five-dimensional per-point representation:

$$p_i = (x_i, y_i, z_i, v_i, I_i) \in \mathbb{R}^5.$$

Moment extraction. For a frame $\mathcal{F}_t = \{p_1, \dots, p_{N_t}\}$, we compute six statistics along each of the $d = 5$ feature dimensions:

Statistic	Definition
Mean	$\mu_k = \frac{1}{N_t} \sum_i p_{ik}$
Std. dev.	$\sigma_k = \sqrt{\frac{1}{N_t} \sum_i (p_{ik} - \mu_k)^2}$
Skewness	$\gamma_{1,k} = \frac{1}{N_t} \sum_i \left(\frac{p_{ik} - \mu_k}{\sigma_k} \right)^3$
Kurtosis	$\gamma_{2,k} = \frac{1}{N_t} \sum_i \left(\frac{p_{ik} - \mu_k}{\sigma_k} \right)^4 - 3$
Minimum	$\min_i p_{ik}$
Maximum	$\max_i p_{ik}$

Concatenating these statistics across all $d = 5$ dimensions yields a fixed-size descriptor

$$s_t = [\mu_1, \sigma_1, \gamma_{1,1}, \gamma_{2,1}, \min_1, \max_1, \dots] \in \mathbb{R}^{30},$$

regardless of the number of points N_t in the frame. This descriptor serves as the input to the classifier described below.

Temporal aggregation. In contrast to the sketching pipeline, which operates over a 2-second window of 60 frames and feeds a $(60, m)$ tensor to a sequential model, the moment baseline treats each frame *independently*: a single frame is compressed into one 30-dimensional vector, which is passed directly to an MLP. This amounts to ignoring the temporal structure of the recording. The implications for performance and a discussion of how temporal information could be reintroduced are given in Section 6.4.

Standardization. The 30-dimensional feature vector is standardized to zero mean and unit variance using statistics computed on the training set only, following the same protocol as in Section 3.1.

6.3 Architecture: Multi-Layer Perceptron

Since the 30-dimensional moment vector has no spatial or sequential structure, convolutional or recurrent architectures bring no benefit here. We therefore use a fully-connected MLP with three hidden layers:

Layer	Output dim	Activation	Note
Input	30	—	Standardized moment vector
Dense 1	128	ReLU	
Dense 2	64	ReLU	
Dense 3	32	ReLU	
Output	5	Softmax	One logit per activity class

The model is trained with early stopping (patience of 10 epochs, validation fraction 15%) to prevent overfitting on the relatively small dataset. No dropout or weight decay was applied, as the model is already severely regularized by the low input dimensionality.

6.4 Results and Discussion

Table 1 reports per-class precision, recall and F1-score on the RadHAR test set (58 samples, 5 classes).

Table 1: Per-class performance of the moment-based MLP on RadHAR (test set).

Activity	Precision	Recall	F1-score	Support
Boxing	0.52	0.85	0.65	13
Jack	0.67	0.20	0.31	10
Jump	0.62	0.45	0.53	11
Squats	1.00	0.20	0.33	10
Walk	0.58	1.00	0.74	14
Overall accuracy		58.6%		

The model achieves **58.6%** accuracy, well above the random baseline of 20%. Locomotor activities with a clear spatial signature (*walk*: 100% recall; *boxing*: 85% recall) are well captured by the moment representation. By contrast, activities involving repetitive vertical motion confined to a small spatial region (*squats*, *jack*) are poorly recalled (20%), likely because a single-frame moment descriptor cannot encode the periodicity that distinguishes these activities from a static standing posture.

This limitation is a direct consequence of the *temporal blindness* of the current approach. Integrating temporal information — for instance by computing moments over a 2-second sliding window rather than per frame, or by feeding the sequence of per-frame descriptors to an LSTM — is the most natural next step and is expected to close a significant part of the gap with the sequential architectures described in Section 3.

7 XGBoost

8 Results

8.1 Comparison

Here we compare everything about our different approaches, F1 and accuracy scores, amount of parameters, Inference time etc.

9 Conclusion

References

A Appendix